

Efficient Route Planning between Stations in NYC: A Dijkstra Algorithm Approach

Alice Nguyen, Prabhleen Gill, Emi Sayuri Gonzales, Emily Greene

April 2024

Abstract

Efficient public transportation is crucial for major cities. The New York City (NYC) Subway is a prime example, transiting on average millions of riderships every week in 2023, and, as such, optimization of transit routes is important. This paper introduces a methodology for optimizing routes within the NYC Subway system, employing Dijkstra's algorithm for computational analysis. Each station is represented as vertices in a weighted graph, with weights corresponding to the true distance between two stations. The results demonstrate significant improvements in route planning efficiency, providing commuters and system planners with a robust tool for enhanced journey planning and network management in one of the world's largest and busiest urban transit systems. The paper explains the algorithm thoroughly, accompanied by a small-scale example for better comprehension. Additionally, coding implementations are included to extend the algorithm's functionality. Limitations of the approach, such as the scaling system used for time and cost, are also discussed.

Table of Contents

Abstract.....	1
Table of Contents.....	2
Introduction.....	3
Dijkstra Algorithm.....	4
A. The Basics and Assumptions.....	4
B. Small Scale Example.....	5
C. Algorithm Limitations.....	10
Application Model to the New York City Subway System.....	11
A. Assumptions.....	11
B. Limitations.....	11
C. The New York City Subway System.....	13
Results.....	14
Conclusion.....	16
Appendix.....	17
Bibliography.....	20

Introduction

Efficient transportation systems are essential for urban environments, and the New York City subway system is no exception. With over 400 stations and numerous interconnected routes, navigating the network effectively is a challenge. The goal of this paper is to utilize graph theory principles, specifically Dijkstra's algorithm, to optimize travel routes within the New York City Subway.

Dijkstra's algorithm was proposed in 1956 by Dutch computer scientist Edsger W. Dijkstra. The purpose of the algorithm is to find the shortest path in weighted graphs, and it has several real-world applications regarding route planning and social networking. This paper will explore how the algorithm works, as well as presenting a model to apply it to the New York City Subway. The first step in accomplishing this goal is to convert the entire subway system into a weighted graph, where each subway station is represented by a node and each line connection between stations is represented by a weighted edge. By using the real-world distance in miles between subway stations as the weights for edges, Dijkstra's algorithm can be applied to find the shortest path between any two subway stations.

While the algorithm is able to find the most efficient path between subway stations solely based on weights, there are several real-world factors that could affect the algorithm's accuracy that are not represented in the graph of New York's subway system alone. Therefore, we will also discuss the limitations of the algorithm. Despite its limitations, the model is a powerful tool with the ability to optimize and improve the experience of billions of commuters.

Dijkstra Algorithm

A. The Basics and Assumptions

Dijkstra's algorithm is designed to find the shortest path between two vertices in a weighted graph. However, its utility extends beyond a single path: it can calculate the shortest distance from one source vertex to all other vertices within the same graph.

Dijkstra's algorithm works by starting at a source vertex and assuming the distance from the source to itself is 0 and to all other vertices is initially infinity. From there, the algorithm explores unvisited connected vertices, updating the shortest known distances to their neighbors. This process repeats until all vertices have been visited and the shortest paths to all reachable vertices are known.

The only requirement is that the graph we are working with has to have positive weights. Dijkstra's algorithm can't handle negative weights because it's based on the assumption that once a vertex is visited with the shortest path locked in, we won't consider that vertex again. Negative weights would violate this principle. If the graph is disconnected, Dijkstra's algorithm will still work. For any vertex that isn't connected to the source, the shortest distance will remain infinity, effectively signaling the lack of a path.

To efficiently track progress during the execution of Dijkstra's algorithm, it is helpful to use a simple table organized into four columns. The first column, *Vertex*, lists all the vertices in the graph. The second column, *Shortest Distance*, keeps track of the minimum known distance from the starting vertex to each vertex as the algorithm progresses. The third column, *Previous Vertex*, records the

preceding vertex on the shortest path, allowing for easy reconstruction of the full route once the algorithm has completed. The last column is used to keep track of the set of all unvisited vertices.

The algorithm proceeds by visiting the unvisited vertex with the smallest known distance at each step and exploring its neighbors. **Here is how the algorithm works step by step:**

Step 1: Select a current vertex: From the set of unvisited nodes, select the node with the smallest distance. This vertex is referred to as the current node. If this is the first iteration, we will select the source node.

Step 2: Update distances to neighbors: For each neighbor of the current vertex, calculate the distance through the current vertex. This is the sum of the current vertex's distance and the weight of the edge connecting it to the neighbor. If the newly calculated distance is strictly smaller than the current recorded distance for the neighbor, update the distance.

Step 3: Mark the current node as visited: After all of its neighbors have been considered, mark the current node as visited. A node is only visited once, and its distance value is never updated again.

Repeat: Continue selecting the unvisited node with the smallest distance as current vertex and repeat step 2 and step 3 until all nodes have been visited or all remaining unvisited nodes are unreachable.

To find the distance of the shortest path from the starting vertex, look for the number in the second column. To find the shortest path, retrace the steps by following the previous steps column and backing up to the starting vertex.

B. Small Scale Example

An example will be carefully explained to help clarify these set-ups and iterations.

Consider an undirected weighted graph below.

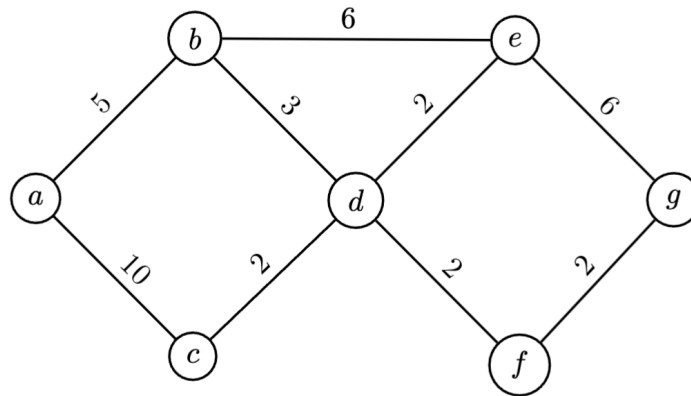


Figure 1: Example

Let's consider the shortest distance between a and g . There is obviously more than one path to go from a to g but applying the Dijkstra Algorithm would find such a path.

To make the process easier, we need some initial setup so that we can keep track of key information as we run the algorithm. We will establish a table to record the shortest distance to each vertex in our graph, along with noting the preceding vertex visited before evaluating the current one.

Visited?	Vertices	Shortest distance from a	Previous vertex
	a	0	
	b	∞	

	c	∞	
	d	∞	
	e	∞	
	f	∞	
	g	∞	

Table 1.1: Example's information table

Before running the algorithm, we have no information about the shortest distance between a and any other vertices, apart from itself. Hence the assignment of 0 to the distance from a and infinity for the other distances.

We start with the vertex that has the shortest distance from the starting vertices, a . In this case, it would be a itself. We will consider a 's neighbor vertices, b and c , and calculate the cost it takes to go from a to b (5 in this case) and from a to c (10). We will update the "Shortest distance from a " column if the new distance is shorter than the current cost. Since 5 and 10 are both less than infinity, we will update the "Shortest distance from a " column of vertex b and c to 5 and 10 respectively, as well as the "Previous vertex" column to a . Finally, mark vertex a as visited and move on to the next lowest cost vertex.

Visited?	Vertices	Shortest distance from a	Previous vertex
X	a	0	
	b	5	a
	c	10	a
	d	∞	
	e	∞	
	f	∞	
	g	∞	

Table 1.2: Example's information table updated

Since b is the unvisited vertex with the shortest distance from a , we will examine b next. b has 3 neighbors— a , d , and e , but we will not consider a since it is a visited vertex. The distance from vertex b to vertex e is 5, so the distance from a to e is $5 + 6 = 11$. Since 11 is less than infinity, it will be updated as the shortest distance from a to e via b . Likewise, the shortest path from a to d would be 8 via b .

Visited?	Vertices	Shortest distance from a	Previous vertex
X	a	0	
X	b	5	a
	c	10	a

	d	8	b
	e	11	b
	f	∞	
	g	∞	

Table 1.3: Example's information table updated

Again, we will look at the unvisited vertex with the lowest distance from a , which is d . The distance from a to c , via d , is $8 + 2 = 10$, which is the same as the distance from a to c directly (as recorded in the table.) Because of this, we do not update the shortest path column for vertex c . However, this tells us that there can be multiple “shortest paths” available. The distance to e is calculated to be $8 + 2 = 10$, which is less than the current distance from a to e , which is 11. The updated distance to f is 10, and the vertex d would be crossed out as visited.

Visited?	Vertices	Shortest distance from a	Previous vertex
X	a	0	
X	b	5	a
	c	10	a
X	d	8	b
	e	11 10	b d

	f	10	d
	g	∞	

Table 1.4: Example's information table updated

The next current vertex is c , but since c has no unvisited neighbors, we consider c visited. When there are two vertices with the same shortest distance from the source, break the tie alphabetically. Repeat the process for the remaining vertices and we should get the following information table as a result.

Visited?	Vertices	Shortest distance from a	Previous vertex
X	a	0	
X	b	5	a
X	c	10	a
X	d	8	b
X	e	11 10	b d
X	f	10	d
X	g	16 12	e f

Table 1.5: Example's information table updated

To find the shortest distance from a to g , look at the g vertex row and you should have a distance of 12. To construct the shortest path from a to g , we can follow the previous vertex of any vertex, all the way to the source. Hence, the shortest path from a to g would be $a - b - d - f - g$.

C. Algorithm Limitations

Dijkstra's algorithm, while powerful, does have its limitations. First and foremost, it fails on graphs with negative weight edges, as the algorithm assumes all weights are non-negative and does not account for potential corrections that such edges might require. Additionally, Dijkstra's algorithm, though effective, is not always the most efficient choice, especially on larger-scale maps. Unlike A*, which employs heuristics to guide its search in the direction of the goal, Dijkstra conducts an exhaustive exploration of all potential routes, resulting in slower performance when dealing with complex or expansive graphs. Studies have consistently highlighted this inefficiency; for instance, a conclusion drawn by Dian Rachmawati and Lysander Gustin (2020) demonstrates that A* can outperform Dijkstra significantly by reducing the number of nodes explored, particularly in scenarios where a goal-oriented approach is beneficial. This contrast underscores Dijkstra's limitation in terms of scalability and computational efficiency in practical applications.

Algorithm Implementation on the NYC Subway System

A. Assumptions

Before diving into the implementation model, we must establish key assumptions about the underlying graph representation of the NYC Subway System used in this paper. The graph is modeled as undirected, which aligns with the system's bidirectional train lines. The nine main lines are interconnected, ensuring that every station in the example is accessible. The graph is an abstraction and not an exact representation of the real system. Certain train lines are merged, and some stations are omitted to streamline the model, as discussed later.

Additionally, transferring between stations or lines is considered cost-free, and edge weights, representing distances in miles, remain constant. The graph is connected, meaning all vertices (stations) are reachable from one another through some path.

Edge weights represent only the physical distance between stations in miles, disregarding factors like transfer costs, platform waiting times, or maintenance delays. These real-world complexities are excluded to simplify the model, as mentioned further in the limitations section. This abstraction allows us to focus purely on distance-based shortest-path calculations.

B. Limitations

As we were collecting data, we made sure to recognize and consider the obstacles and limitations we would encounter. Due to the dynamic nature of urban infrastructure it was challenging to predict any changes the subway station could face, as those factors were beyond our control. We

recognize our data may not be as accurate and up to date for these reasons, and that it is crucial to review these challenges to ensure our data is still able to accurately represent the subway network.

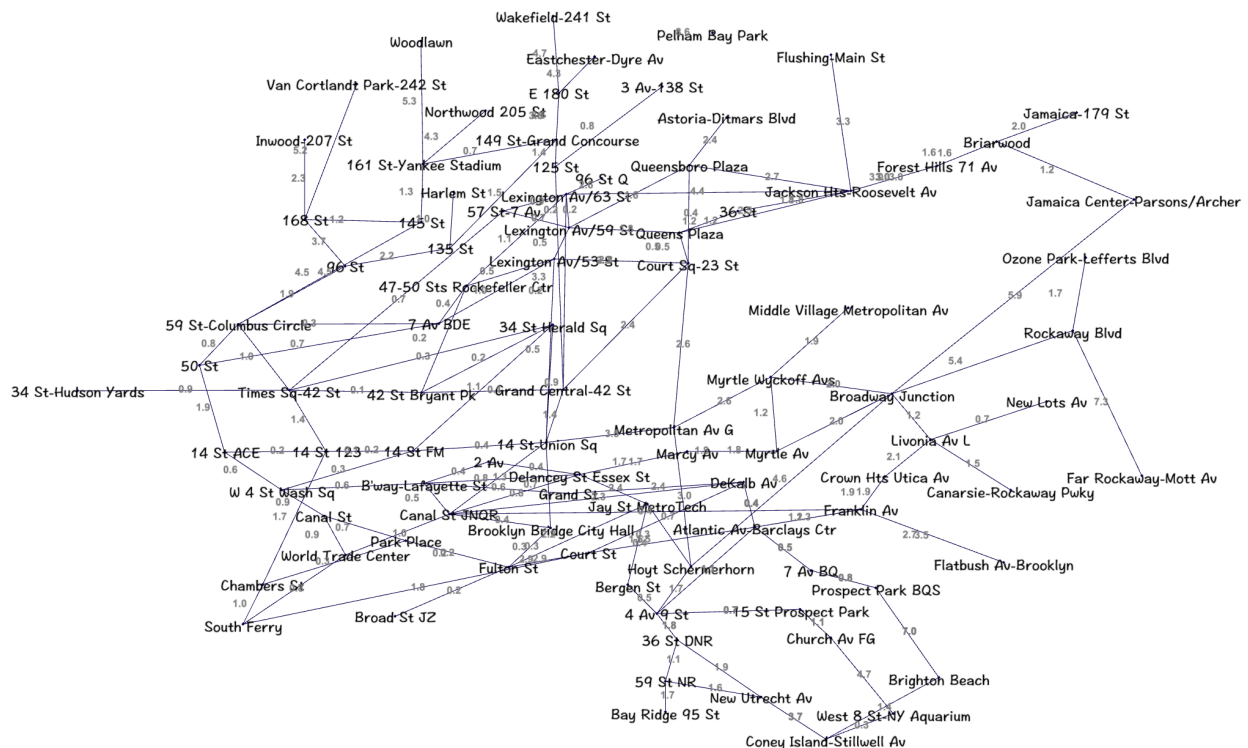
One challenge we encountered in our study was the possibility of temporary closures of the stations. These closures could be caused by the occurrence of unforeseen events such as construction, maintenance, or emergencies. Though we aimed to provide the shortest distance between two stations, these unpredictable events make it challenging for it to happen as rerouting may result in larger distances.

It was also challenging to consider cases where subway stations could become inaccessible. A station may be overcrowded on certain days and at certain times. This allows for the possibility of missing rides and needing to look for a new route or wait for the ride to return, a process that would lead to more consumption of time and the possibility of rerouting to another station farther away. Individuals with disabilities may not be able to access the subway in a more timely manner than individuals who do not have disabilities, stressing the fact that individuals are also variables in determining the most efficient route. Some subway stations have accommodations and easier access for those with mobility disabilities, but they won't be specified in the graph. Weather conditions and geographical barriers may prevent individuals from reaching stations on time and may put stations at a higher risk of closures.

C. Graph Translation of the NYC Subway System

The algorithm will be applied to the NYC Subway System following the main 9 lines, but the subway system must first be converted into a graph. To convert the subway system into a graph, this

paper will utilize a free program called Gephi. This means that each piece of data needs to be inputted manually which may lead to human error.



Note that some edges may have multiple subway lines that go along that edge with different weights. The weights were calculated by finding the distance between the “source station” and “target station” in miles. The cost will be calculated based on the distance in miles and not the cost of entry because the monetary cost of entry is a flat rate of \$2.90 with some exceptions.

Results

The Python program was able to generate the shortest path and calculate the total distance between two stations using Dijkstra’s algorithm. Additionally, a graph was developed using the data of the subway system and the shortest path is indicated by a red line.

```
Welcome to the NY Subway System Route Finder!

Do you want to see the list of the stations (YES/NO): yes

Here is a list of stations:
- 125 St
- 135 St
- 14 St 123
- 14 St ACE
- 14 St FM
- 14 St-Union Sq
- 145 St
- 149 St-Grand Concourse
- 15 St Prospect Park
- 161 St-Yankee Stadium
- 168 St
- 2 Av
- 3 Av-138 St
- 34 St Herald Sq
- 34 St-Hudson Yards
- 36 St
- 36 St DNR
- 4 Av 9 St
- 42 St Bryant Pk
- 47-50 Sts Rockefeller Ctr
```

```

Enter a source station: Park Place
Enter a destination station: Grand St

We found the following best path with a distance of 2.0 miles.
Park Place -> Fulton St -> Brooklyn Bridge City Hall -> Canal St JNQR -> B'way-Lafayette St -> Grand St

Do you want to view the visualization of the graph (YES/NO) ? |

```

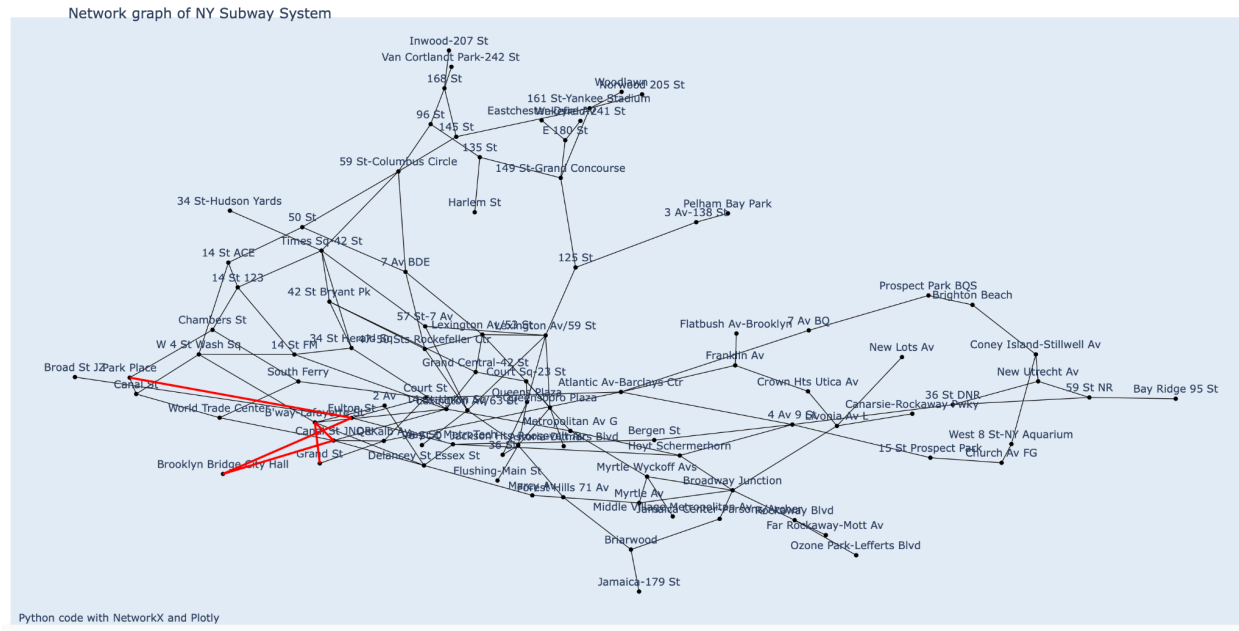


Figure 3: Output of Python Program

Conclusion

Efficient public transportation is vital for urban life, and this study applied Dijkstra's algorithm to optimize route planning within the NYC Subway. By modeling stations and connections as a weighted graph based on physical distances, we demonstrated the algorithm's ability to identify optimal routes. While the model simplifies real-world complexities, it offers a strong foundation for improving commuter experiences.

A possible next step is to develop an updated model incorporating additional factors like delays, accessibility, or crowding. Additionally, designing a user-friendly phone app could bring this optimization directly to commuters, enhancing usability and real-time decision-making. This approach could revolutionize transit systems globally.

Appendix

In support of our study of the NY subway network, a custom Python program was developed to perform the necessary computations and to visualize the subway network. Due to the extensive and complex character of the subway network, it was crucial to have a program that would run Dijkstra's algorithm and determine the shortest route between two stations.

The station names and weights, which were the distances between two stations in miles, were stored in two separate containers: one for the names of the stations, and one for the distance between two stations. Using these pieces of data as well as a supporting function, a graph was able to be developed within the system.

```

nodes = ["Van Cortlandt Park-242 St", "168 St", "96 St", "59 St-Columbus Circle", "Times Sq-42 St", "14 St 123",
"Chambers St", "Chambers St", "Park Place", "Fulton St", "Court St", "Atlantic Av-Barclays Ctr", "Franklin Av",
"Flatbush Av-Brooklyn", "Crown Hts Utica Av", "Livonia Av L", "New Lots Av", "135 St", "Harlem St",
"149 St-Grand Concourse",
"E 180 St", "Wakefield-241 St", "Inwood-207 St", "145 St", "50 St", "7 Av BDE", "Lexington Av/53 St",
"Court Sq-23 St",
"Queens Plaza", "Jackson Hts-Roosevelt Av", "Forest Hills 71 Av", "Briarwood", "14 St ACE", "W 4 St Wash Sq",
"Canal St",
"Jay St MetroTech", "Hoyt Schermerhorn", "Broadway Junction", "Rockaway Blvd", "Ozone Park-Lefferts Blvd",
"Far Rockaway-Mott Av", "Woodlawn", "161 St-Yankee Stadium", "Eastchester-Dyre Av", "125 St", "3 Av-138 St",
"Pelham Bay Park", "Lexington Av/59 St", "Grand Central-42 St", "14 St-Union Sq", "B'way-Lafayette St",
"Canal St JNQR",
"Brooklyn Bridge City Hall", "Norwood 205 St", "47-50 Sts Rockefeller Ctr", "Lexington Av/63 St",
"Jamaica-179 St",
"36 St", "42 St Bryant Pk", "34 St Herald Sq", "14 St FM", "Delancey St Essex St", "Marcy Av", "Myrtle Av",
"Myrtle Wyckoff Avs", "Middle Village Metropolitan Av", "2 Av", "Bergen St", "4 Av 9 St",
"15 St Prospect Park",
"Church Av F6", "West 8 St-NY Aquarium", "Grand St", "DeKalb Av", "36 St DNR", "New Utrecht Av", "7 Av BQ",
"Prospect Park BQS", "Brighton Beach", "34 St-Hudson Yards", "Queensboro Plaza", "Astoria-Ditmars Blvd",
"Flushing-Main St",
"96 St Q", "57 St-7 Av", "World Trade Center", "South Ferry", "59 St NR", "Bay Ridge 95 St",
"Coney Island-Stillwell Av",
"Metropolitan Av 6", "Canarsie-Rockaway Pkwy", "Broad St JZ", "Jamaica Center-Parsons/Archer"]

```

```

connections = [("Van Cortlandt Park-242 St", "168 St", 5.2),
("168 St", "96 St", 3.7), ("96 St", "59 St-Columbus Circle", 1.9),
("59 St-Columbus Circle", "Times Sq-42 St", 1), ("Times Sq-42 St", "14 St 123", 1.4),
("14 St 123", "Chambers St", 1.7),
("Chambers St", "South Ferry", 1), ("Chambers St", "Park Place", 0.3), ("Park Place", "Fulton St", 0.2),
("Fulton St", "Court St", 2.9), ("Court St", "Atlantic Av-Barclays Ctr", 1.5),
("Atlantic Av-Barclays Ctr", "Franklin Av", 1.2),
("Franklin Av", "Flatbush Av-Brooklyn", 2.7), ("Franklin Av", "Crown Hts Utica Av", 1.9),
("Crown Hts Utica Av", "Livonia Av L", 2.1),
("Livonia Av L", "New Lots Av", 0.7), ("96 St", "135 St", 2.2), ("135 St", "Harlem St", 1),
("135 St", "149 St-Grand Concourse", 1.5),
("149 St-Grand Concourse", "E 180 St", 3.6), ("E 180 St", "Wakefield-241 St", 4.7),
("Inwood-207 St", "168 St", 2.3), ("168 St", "145 St", 1.2),
("145 St", "59 St-Columbus Circle", 4.5), ("59 St-Columbus Circle", "50 St", 0.8),
("50 St", "7 Av BDE", 0.7),
("7 Av BDE", "Lexington Av/53 St", 1), ("Lexington Av/53 St", "Court Sq-23 St", 2.3),
("Court Sq-23 St", "Queens Plaza", 0.5),
("Queens Plaza", "Jackson Hts-Roosevelt Av", 2.5), ("Jackson Hts-Roosevelt Av", "Forest Hills 71 Av", 3),
("Forest Hills 71 Av", "Briarwood", 1.6),
("Briarwood", "Jamaica Center-Parsons/Archer", 1.2),
("50 St", "14 St ACE", 1.9),
("14 St ACE", "W 4 St Wash Sq", 0.6),
("W 4 St Wash Sq", "Canal St", 0.9),
("Canal St", "World Trade Center", 0.9),
("Canal St", "Park Place", 0.7),
("Park Place", "Fulton St", 0.2),
("Fulton St", "Jay St MetroTech", 2.2),
("Jay St MetroTech", "Hoyt Schermerhorn", 0.3),

```

Figure 4: Storage of Station Data in Containers

To calculate the shortest distance, Dijkstra's algorithm was implemented within the program. The algorithm took the graph and the starting station, which was chosen by user input, and ran a breadth-first search based on the weights. As each station was visited the station's name and the tentative weight were stored in separate containers.. Once the algorithm was completed, it would return a path for the shortest route as well as all of the nodes contained in the path.

```
def dijkstra_algorithm(graph, start_node):
    unvisited_nodes = list(graph.get_nodes())
    shortest_path = {}
    previous_nodes = {}

    max_value = sys.maxsize
    for node in unvisited_nodes:
        shortest_path[node] = max_value
    shortest_path[start_node] = 0

    while unvisited_nodes:
        current_min_node = None
        for node in unvisited_nodes:
            if current_min_node == None:
                current_min_node = node
            elif shortest_path[node] < shortest_path[current_min_node]:
                current_min_node = node

        neighbors = graph.get_outgoing_edges(current_min_node)
        for neighbor in neighbors:
            tentative_value = shortest_path[current_min_node] + graph.value(current_min_node, neighbor)
            if tentative_value < shortest_path[neighbor]:
                shortest_path[neighbor] = tentative_value
                previous_nodes[neighbor] = current_min_node

        unvisited_nodes.remove(current_min_node)

    return previous_nodes, shortest_path
```

Figure 5: Dijkstra's Algorithm as a Python Function

Upon running the program, the user was greeted with a welcome message and was prompted with an option to see the list of stations. If the user said "yes", the list of stations was printed and the user was then asked to select the source station and the destination station. The program produced and displayed the shortest path at the user's convenience. At the end, the user could choose to view the

visualization of the graph, and upon saying “yes”, the graph of the subway network along with the shortest route was displayed. This graph was developed using Plotly and Networkx, packages that help with developing and visualizing graphs in Python. The graph depicted the network of stations and indicated the shortest route between two stations.

Input: *graph, start_node*

Output: *previous_nodes, shortest_path*

for *each unvisited node* **do** *shortest_path[node] = ∞ ; shortest_path[start_node] = 0*

while *all nodes are unvisited* **do** *current_min_node = none*

for *each unvisited node* **do**

case 1. *current_min_node == none; current_min_node = node*

case 2. *shortest_path[node] < shortest_path[current_min_node]; current_min_node = node;*

for *each neighbor* **do** *tentative_value = shortest_path[current_min_node] + graph.value(current_min_node, neighbor)*

case 1. *tentative_value < shortest_path[neighbor];*

shortest_path[neighbor] = tentative_value

previous_nodes[neighbor] = current_min_node;

remove current_min_node from list of unvisited nodes

return *previous_nodes, shortest_path*

Table 2: Pseudocode of Dijkstra’s Algorithm

Bibliography

- Filbert. *Efficient Route Mapping in the Singapore MRT Leveraging Graph Theory and Dijkstra's Algorithm*, no. Institut Teknologi Bandung, Jl. Ganesha 10 Bandung 40132, Indonesia., 2023.
[https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2023-2024/Makalah2023/Makalah-Matdis-2023%20\(21\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2023-2024/Makalah2023/Makalah-Matdis-2023%20(21).pdf). Accessed 15 April 2024.
- Rachmawati, Dian, and Lysander Gustin. "Analysis of Dijkstra's Algorithm and A* Algorithm in Shortest Path Problem." *Journal of Physics: Conference Series*, vol. 1566, no. November 2019, 2020, pp. 26-27.
- Schauer, Travis. "Network Analysis of the Paris and Tokyo Subway Systems." no. YOUNGSTOWN STATE UNIVERSITY.
https://etd.ohiolink.edu/acprod/odb_etd/ws/send_file/send?accession=ysu1682612823083866&disposition=inline. Accessed 15 April 2023.